Static Plan or Dynamic Dispatch? Task Scheduling on a Hybrid CPU/GPU Machine

Project demonstration: existing implementation versus proposed work

Lakshay S (26MAI0064), Arkkadu Bharath Reddy (26MAI0058)

Computer Architecture and Organization | Guide: Dr. Yokesh Babu S, Associate Professor Grade 1

Measured on one laptop (RTX 3050 + P/E cores); remote worker emulated; Raspberry Pi not tested

What problem are we solving?

- A workflow is a graph of tasks with data dependencies (here 64 image chains, 257 tasks; earlier also the Montage mosaic).
- The machine is heterogeneous: 6 P cores, 7 E cores (2.8 to 3.6x slower for these tasks) and one CUDA GPU worker (RTX 3050).
- A static scheduler (HEFT, PEFT) plans everything before the run; a dynamic scheduler decides while running.
- Research question: with ONE cost model on both sides, is dynamic faster than static once planning and start-up time are counted, and when?
- Blind placement alone costs 17% to 40% more time than a cost-aware dynamic policy when task cost varies.

Existing implementation: the base idea (B4)

- Base patent US 11,561,826 B1 (Xilinx): task graph, kernel objects tied to compute circuits, per-kernel queues, in-degree gating.
- Mapped in our runtime as B4: a task goes to the first idle worker it is eligible for (P before E). No estimate of execution time.
- Strength: keeps every worker busy, about 7 ms of decision time per run.
- Weakness: order is first come first served and the worker is almost never chosen (93.6% of dispatches had one eligible idle worker); blind to task cost.
- On the GPU configurations blind B4 sends about a third of its tasks to the GPU and is slower for it (+1.6%, inconclusive).

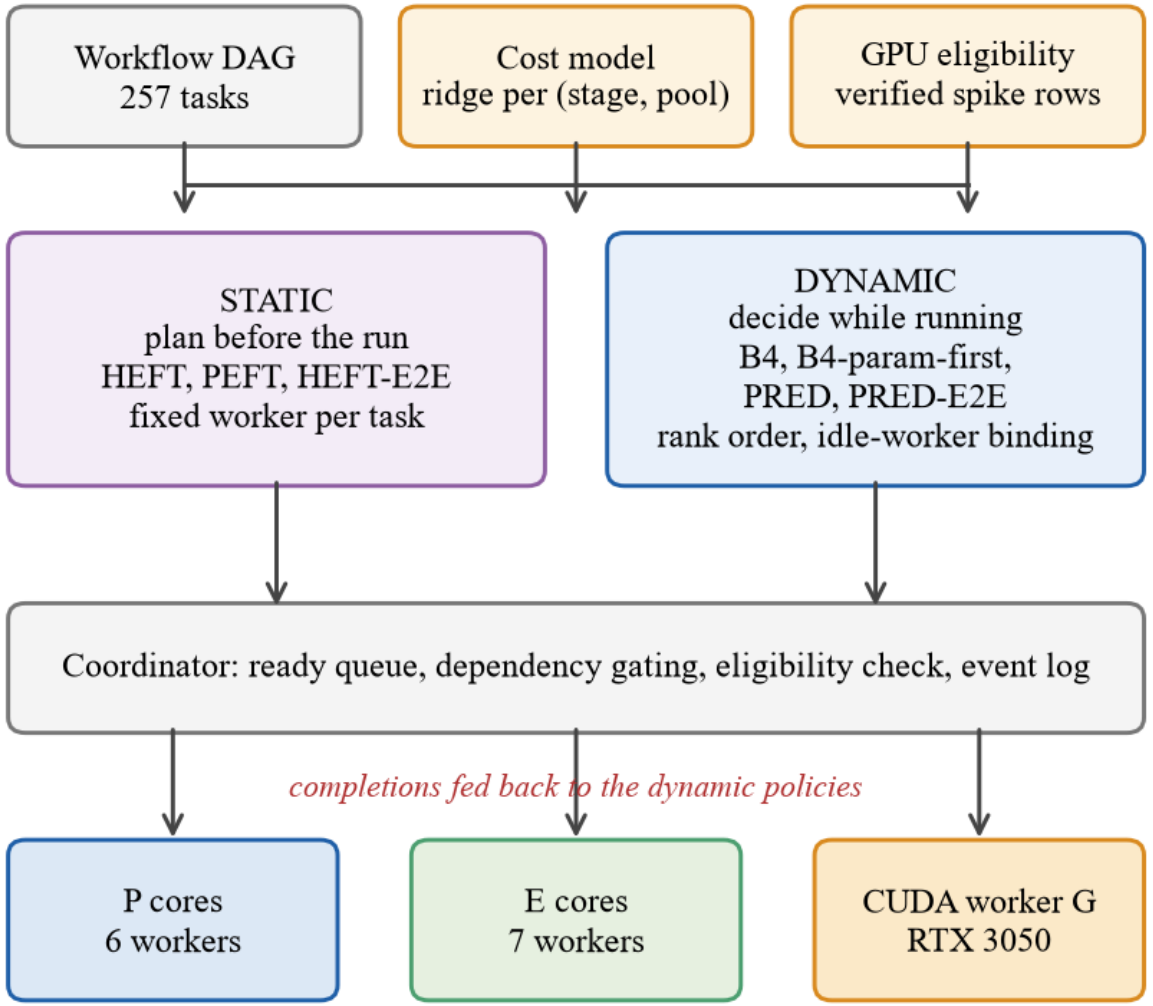
Proposed work

- Dynamic: order ready tasks by predicted upward rank; bind only to an IDLE eligible worker with the least predicted time (PRED_RANK_GREEDY); an end-to-end variant adds set-up and IPC terms.
- Static: HEFT, PEFT and HEFT_E2E plans from the SAME ridge model, computed before the run; an oracle plan as a reference only.
- GPU eligibility per task from recorded speed-up measurements (strict and extended sets); no configuration is GPU-only.
- End-to-end makespan: start of the run to the last completion, so planning and start-up are counted.
- Protocol frozen before the evaluation; ten pre-stated contrasts with Holm correction; six workloads.

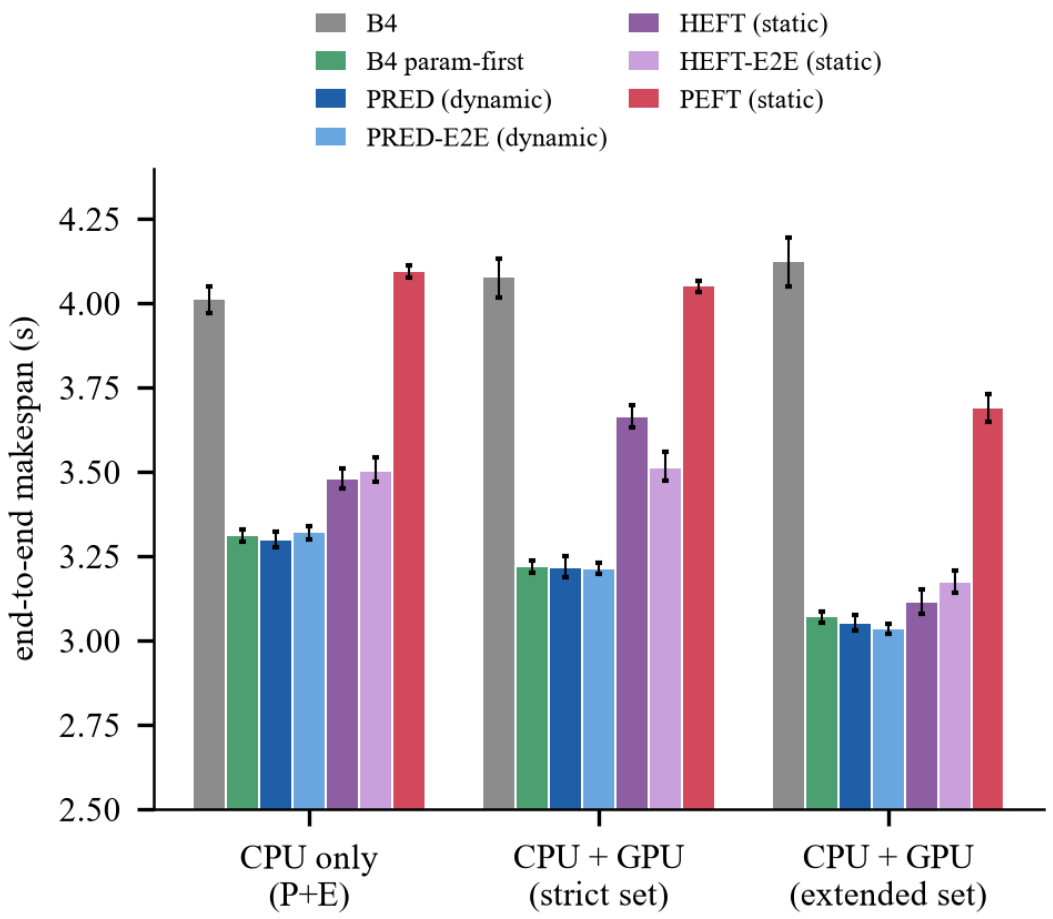
Existing versus proposed at a glance

Aspect	Existing (B4)	Proposed
Order of ready tasks	first come, first served	predicted upward rank
Worker choice	first idle eligible (P before E)	idle eligible, least predicted time; GPU only for verified stages
Cost knowledge	none	ridge model from logs, one-command refit
Primary workload, CPU	reference	−17.8% [−18.8, −16.7] vs B4; −5.2% vs static HEFT
Primary workload, strict GPU	reference	−21.2% vs B4; −12.2% vs static HEFT
No spread in task cost	reference	+0.0% (equivalent within 2%)
Versus a rule that knows the parameter	-	within 1% in every configuration

Architecture of the final framework

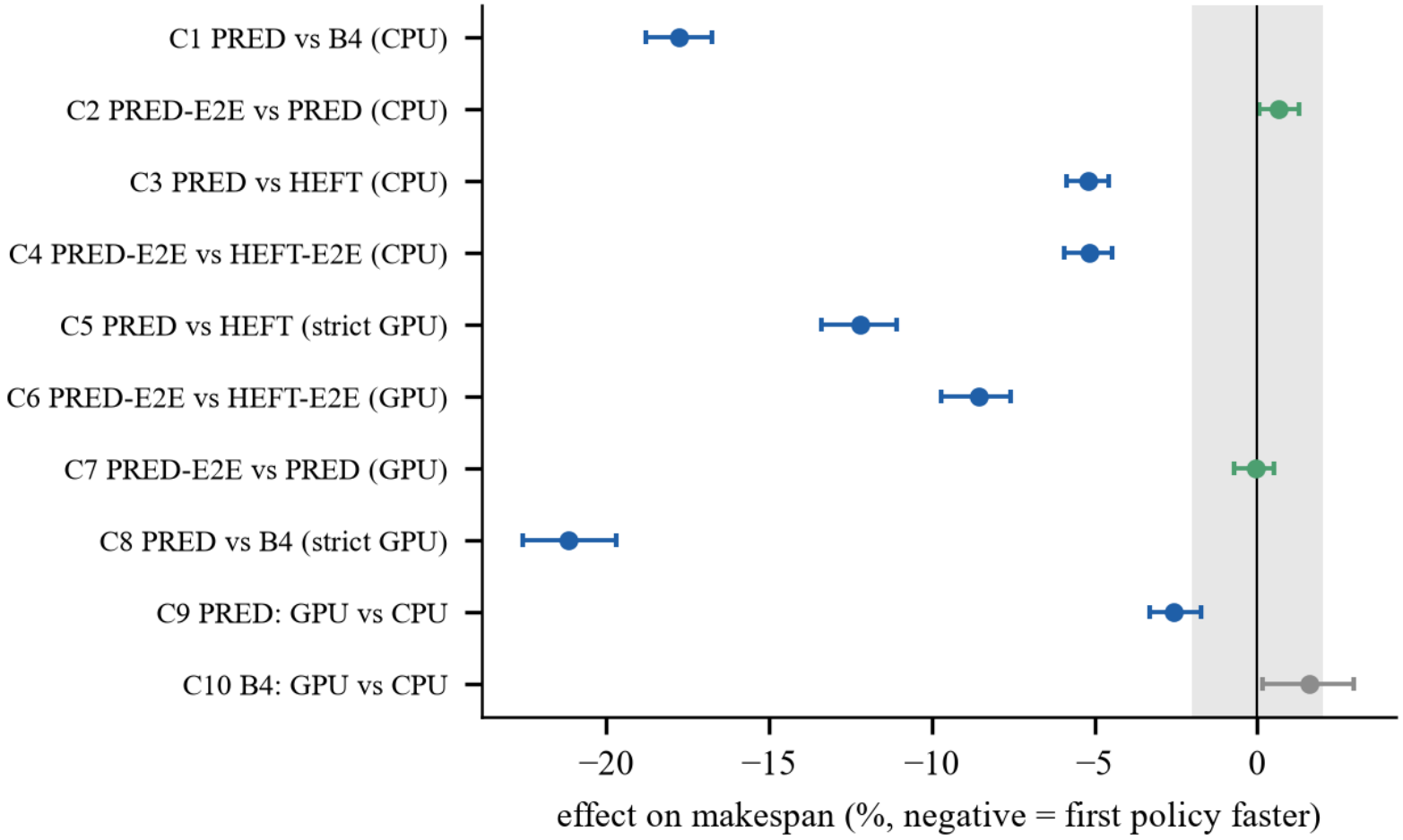


Headline result: primary workload, all policies



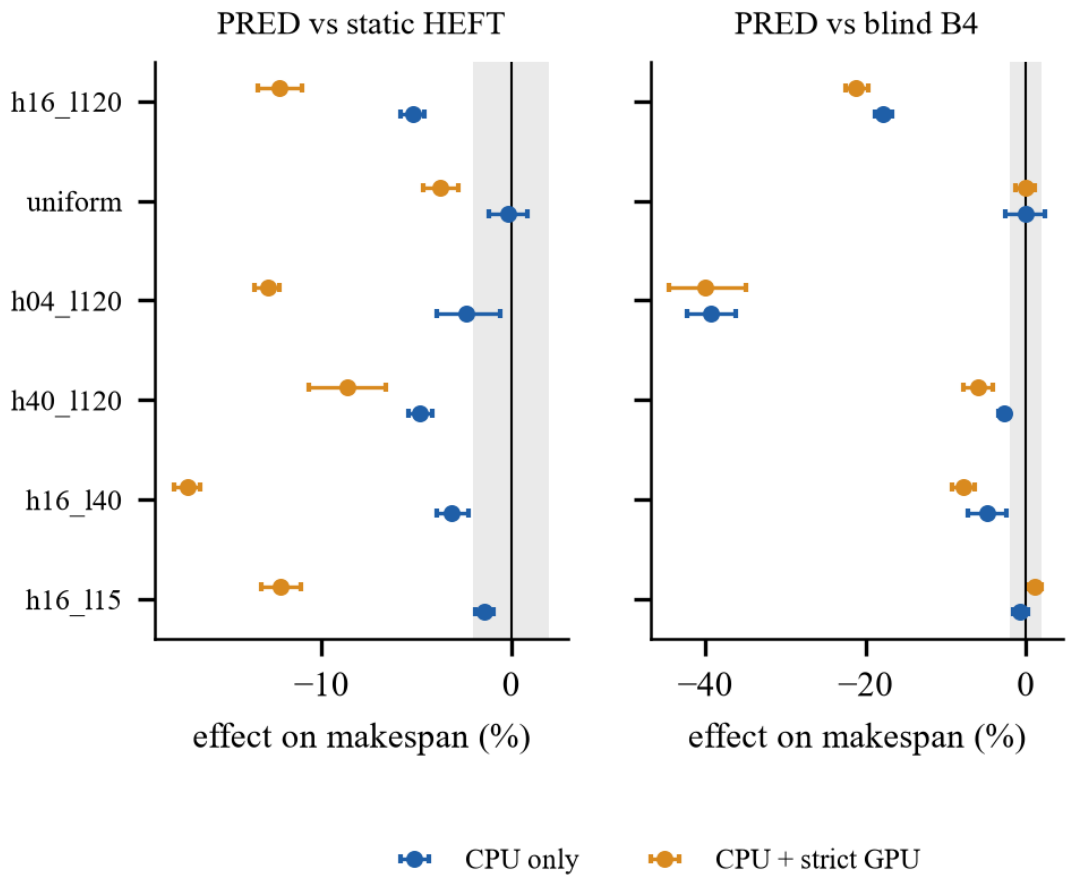
End-to-end makespan, geometric mean over 24 blocks, 95% intervals. B4 param-first is a hand rule that knows the cost parameter.

Ten pre-stated contrasts (Holm-adjusted)



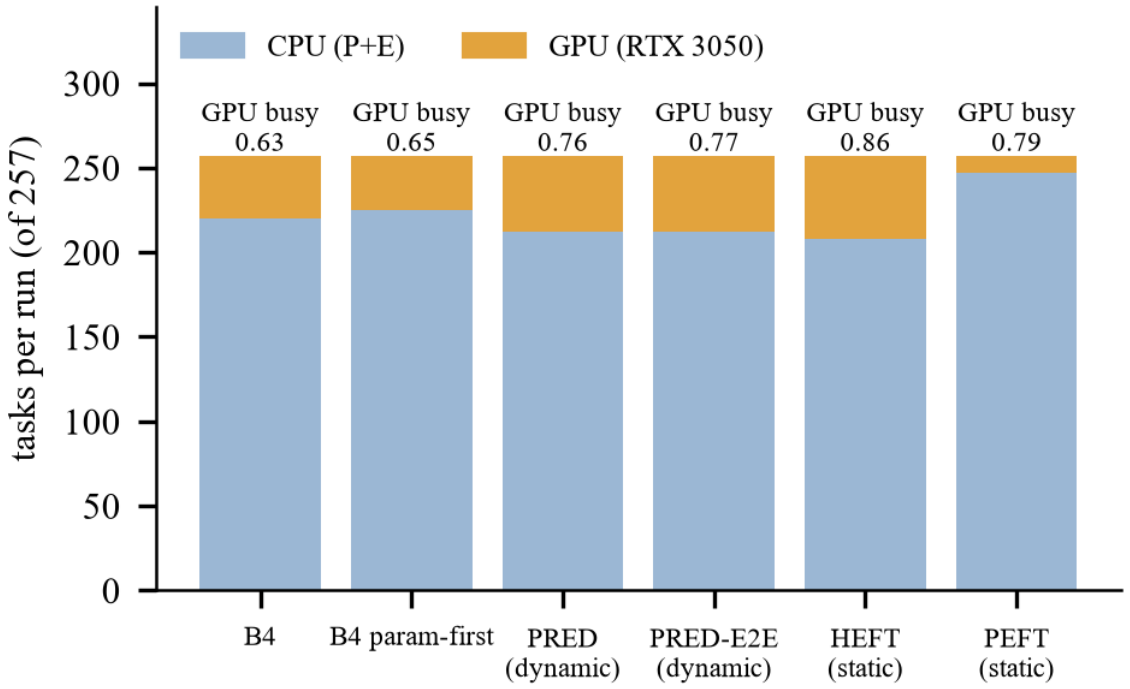
C3/C4 were predicted inconclusive; the dynamic policy was 5% faster, so that prediction was wrong (reported).

When does it help? Six workloads



About 40% over B4 with 4 heavy chains; 3% to 8% with 40 heavy or a cost ratio of 20; nothing on the uniform workload.

The GPU: where it helps and where it does not



- PRED: strict set 2.6% faster than CPU only, extended set 7.5%; blind B4: 1.6% slower with the strict set.
- GPU cost model error is 23% to 31% (CPU: 4% to 11%): the weakest part; limits GPU placement and may inflate the dynamic-vs-static gap.
- One CUDA worker; the GPU takes about 15% to 20% of the 257 tasks under the model-based policies.

Where it does not help (reported honestly)

- A hand rule that knows the cost parameter is within 1% of the dynamic dispatcher in every configuration.
- No spread in task cost (uniform, light workloads): policies tie; with ratio 7.5 and the strict GPU set PRED is +1.1% slower than B4.
- End-to-end cost terms change nothing (equivalent within 2%); a hybrid B4/PRED selector would gain at most 1.1%: not built.
- An earlier session found HEFT level with the dispatcher (+1.10%); the final session finds PRED 5% faster; the difference was not isolated.
- Remote worker: emulation only, no benefit; no Raspberry Pi or real-network result. Montage: no gain.

Demo: front end (dashboard v3)

Command (project root):

```
.\.venv\Scripts\python.exe -m caosched.dashboard_v3 --port 8770 --events-dir data\results\final_20261101\eval\sessions --open
```

- Screen 1: GPU CONNECTED with live load, memory, temperature and power; Raspberry Pi card: NOT CONNECTED.
- Screen 3: final results with hover details (makespan, forest plot, six workloads, task placement).
- Screen 6: run one live block of the final benchmark with any policy on CPU or CPU+GPU (about 1 to 2 minutes; needs 2.4 GB free RAM).
- The original dashboard and v2 stay unchanged (ports 8765 and 8766).

Demo: terminal

- Tests: .\.venv\Scripts\python.exe -m pytest tests -q (520 of 521 pass; the one failure is a RAM-gate test that refuses to start below 2 GB free).
- Final report: CAO_FINAL_EXPERIMENT_REPORT.md, generated from data\results\final_20261101\eval\analysis_*; flat table RESULTS_TABLE_FINAL.csv.
- Source: src\caosched\final\ (static baselines, E2E dispatcher, GPU region), core\, study2\, estimators\, experiment\, analysis\.
- If PowerShell blocks Activate.ps1: do not change the policy; call .venv\Scripts\python.exe directly.

Evidence and integrity

- Every included run's output verified against a CPU reference; event-log and design hashes match the manifests.
- Protocol and design hashes logged before the first evaluation session; calibration, hold-out and evaluation data are separate.
- RAM gate (2.4 GB start, 0.8 GB per block) and mains power enforced before every block; thresholds never lowered; aborted attempts kept, never pooled.
- Earlier results untouched: RESULTS_TABLE.csv unchanged; new table is RESULTS_TABLE_FINAL.csv; backups of every edited file.
- Limits: one machine, one workload family, one CUDA worker, emulated remote.

Next steps and publication

- A GPU cost model with lower error; an experiment that separates model error from decision timing; a second workload family.
- Remote worker with the final policies, then a Raspberry Pi 5 over a wired link (runbook ready, loopback rehearsal only).
- Venue: CCGrid 2027 (abstract 24 Nov, paper 1 Dec 2026, 10 pages) or a Q1 journal (FGCS, IEEE TPDS) with the extended version.
- IPR: file the disclosure before any public release; professional prior-art search first (an IBM grant discloses learned rank-ordered dispatch).